Шаблон технического задания (SPEC.md)

Ниже — формализованное содержание. Всё, что в угловых скобках <...>, студент заменяет своим. Курсивом — инструкции, которые при сдаче удаляются. Примеры приведены для воздушной стихии.

Шапка документа

```
# ТЗ: <Название транспорта>

Автор: <ФИО>
Стихия: <земля | вода | воздух>
Проект: <название>
Репозиторий:<ссылка на публичный репозиторий>
Дата заморозки: <дата>
Версия: <1.0>
```

Инструкция: версия меняется только после заморозки, при правках через CHANGELOG.md. До заморозки версия всегда 0.x.

1. Назначение

Инструкция: 2–4 предложения. Что это за транспорт, для чего он, чем отличается от обычного. Без маркетинга, только суть.

Пример:

> Летающий крокодил «Гроза» — гибридное средство передвижения, сочетающее биологический машущий полёт с реактивным ускорением. Предназначен для перемещения одного пилота над пересечённой местностью. Способен кратковременно нырять в воду.

Крайне желательно визуализировать аппарат. Понятно, что на картинке будет больше деталей, чем удастся описать в ТЗ, но это позволит запомнить работу и основные характеристики средства передвижения.



2. Стихия и конструкционный элемент

Инструкция: обязательный раздел. Стихия задаёт среду. Конструкционный элемент — обязательная деталь, которую нужно обыграть. Без него проект не принимается.

Стихия: воздух.

Конструкционный элемент: не менее одного элемента, обеспечивающего подъёмную силу.

Как реализован элемент:

> Машущие крылья (атрибут `wing_span_m`), создающие подъёмную силу. При отказе крыльев транспорт не может летать.

Ограничения стихии:

- Основной режим — полёт.
- Может кратковременно погружаться в воду, но не живёт там.
- Не предназначен для длительного движения по земле.

---

3. Публичный интерфейс

Инструкция: точные сигнатуры. Разработчик реализует ровно это. Всё, что не описано здесь, — внутренняя реализация, её можно менять.

3.1. Класс

```
class <ClassName>:
```

Имя класса: FlyingCrocodile

Модуль: flying_crocodile.py

3.2. Конструктор

```
def __init__(
    self,
    name: str,
    mass_kg: float,
    wing_span_m: float,
    max_speed_kmh: float,
    energy: float,
) -> None:
```

Параметр	Тип	Единица	Допустимый диапазон	Значение по умолчанию
name	str	—	непустая строка	—
mass_kg	float	кг	> 0	—
wing_span_m	float	м	> 0	—
max_speed_kmh	float	км/ч	> 0	—
energy	float	условные единицы	0 … 100	100.0

Исключения конструктора:

- ValueError — если name пустой.
- ValueError — если mass_kg, wing_span_m или max_speed_kmh ≤ 0.

- `ValueError` — если `energy` вне диапазона 0…100.

3.3. Атрибуты

Инструкция: только публичные. Приватные не описываются — они внутренняя реализация.

Атрибут	Тип	Единица	Доступ	Описание
`name`	str	—	чтение	Имя транспорта
`mass_kg`	float	кг	чтение	Масса
`wing_span_m`	float	м	чтение	Размах крыльев
`max_speed_kmh`	float	км/ч	чтение	Максимальная скорость
`energy`	float	условные единицы	чтение	Текущий запас энергии
`position_km`	float	км	чтение	Пройденное расстояние
`is_flying`	bool	—	чтение	В воздухе ли транспорт

3.4. Методы

Инструкция: для каждого метода — сигнатура, назначение, параметры, возврат, исключения, изменение состояния.

3.4.1. `move`

```
def move(self, distance_km: float, terrain: str) -> float:
```

Назначение: перемещает транспорт на заданное расстояние в заданной среде.

Параметры:

- `distance_km` — расстояние в км, > 0.
- `terrain` — среда: `"air"`, `"water"`, `"ground"`.

Возвращает: время перемещения в часах (float).

Изменяет: `position_km`, `energy`, `is_flying`.

Исключения:

- `ValueError` — если `distance_km` ≤ 0.
- `ValueError` — если `terrain` не из списка.
- `RuntimeError` — если энергии недостаточно (должно быть заменено, см. раздел 6).

Правила:

- В воздухе скорость = `max_speed_kmh`.
- В воде скорость = `max_speed_kmh * 0.5`.
- На земле транспорт не может двигаться: `RuntimeError`.
- Энергия тратится: `distance_km * 2` в воздухе, `distance_km * 1` в воде.
- `is_flying` = True, если `terrain == "air"`, иначе False.

3.4.2. `can_overcome`

```
def can_overcome(self, obstacle: str) -> bool:
```

Назначение: проверяет, может ли транспорт преодолеть препятствие.

Параметры:

- `obstacle` — тип препятствия: `"wall"`, `"river"`, `"tree"`.

Возвращает: `True` / `False`.

Исключения:

- `ValueError` — если `obstacle` не из списка.

Правила:

- `"wall"` — только если `is_flying`.
- `"river"` — только если транспорт в воде или может нырнуть (energy > 20).
- `"tree"` — только если `wing_span_m` > 5.

3.4.3. `calculate_energy`

```
def calculate_energy(self, distance_km: float, terrain: str) -> float:
```

Назначение: рассчитывает затраты энергии без изменения состояния.

Параметры: те же, что у `move`.

Возвращает: затраты энергии (float).

Исключения: те же, что у `move`, кроме `RuntimeError`.

Правила: формула совпадает с `move`, но состояние не меняется.

3.4.4. `refuel`

```
def refuel(self, amount: float) -> None:
```

Назначение: пополняет энергию.

Параметры:

- `amount` — количество энергии, > 0.

Возвращает: `None`.

Изменяет: `energy`, но не выше 100.

Исключения:

- `ValueError` — если `amount` ≤ 0.

3.4.5. `get_status`

```
def get_status(self) -> dict:
```

Назначение: возвращает снимок состояния.

Возвращает: словарь с ключами:

- "name" — str,
- "energy" — float,
- "position_km" — float,
- "is_flying" — bool,
- "wing_span_m" — float.

---

4. Сценарии

4.1 Модульные тесты

*Инструкция: 3–4 сценария. Каждый — шаги и ожидаемый результат. Это основа для **модульных** тестов. Где применяется паттерн AAA.*

Сценарий 1. Обычный полёт

Дано: крокодил с energy = 100, max_speed_kmh = 60.
Когда: move(30, "air").
Тогда:

- возвращается 0.5 (30 / 60);
- position_km == 30;
- energy == 40 (100 − 30 × 2);
- is_flying == True.

Сценарий 2. Полёт с нехваткой энергии

Дано: energy = 10, max_speed_kmh = 60.
Когда: move(30, "air").
Тогда: RuntimeError. (приведено базовое исключение, но оно должно быть заменено! см. раздел 6)

Сценарий 3. Препятствие

Дано: крокодил в полёте (is_flying = True).
Когда: can_overcome("wall").
Тогда: True.

Сценарий 4. Заправка

Дано: energy = 20.
Когда: refuel(50).
Тогда: energy == 70.

4.2 Функциональные (сценарные) тесты

*Инструкция: 3–4 сложных сценария. Каждый — шаги и ожидаемый результат. Это основа для **функциональных**|**сценарных** тестов. Где применяется паттерн GWT. Given-When-Then (GWT) — это способ записи сценария, при котором тест читается как история:*
Given — дано: начальное состояние мира.
When — когда: действие или событие.
Then — тогда: ожидаемый результат.
Один сценарий может содержать несколько блоков When-Then, если история состоит из шагов.

(Markdown шаблон)

```
### Сценарий: <название>

**Given:** <начальное состояние мира>

**When:** <действие>

**Then:** <ожидаемый результат>

**And:** <дополнительный результат>

**When:** <следующее действие>

**Then:** ...
```

Сценарий 5. Гонка над Лавовым ущельем

```
### Сценарий: Гонка над Лавовым ущельем

Given крокодил «Гроза» с массой 500 кг, размахом крыльев 8 м,
      максимальной скоростью 60 км/ч и энергией 80
And   крокодил находится на земле

When  пилот командует взлёт на 10 км
Then  крокодил в воздухе
```

```
And   энергия снижается до 60
And   пройдено 10 км

When  пилот летит ещё 20 км, попадая в бурю
Then  скорость падает вдвое, до 30 км/ч
And   время полёта равно 20 / 30 ч
And   энергия снижается до 20

When  пилот пытается преодолеть скалу
Then  крокодил может её преодолеть
And   энергия снижается до 5

When  пилот пытается лететь ещё 5 км на критическом уровне энергии
Then  поднимается CrocodileNoStrength
And   позиция не меняется

When  пилот командует экстренное снижение в воду на 5 км
Then  крокодил плывёт со скоростью 30 км/ч
And   энергия снижается до 0
And   крокодил не в воздухе

When  пилот заправляется на 50 единиц
And   летит 3 км
Then  энергия равна 44
And   пройдено 38 км
And   крокодил в воздухе
```

Инструкция: при составлении сложных сценариев стоит тесты вести в рамках бизнес-логики. Один из способов — это ввести объект (в примере «пилот»), который управляет объектом. Можно черпать вдохновение в объекте client, который доступен в unittest фреймворка Джанго.

Сценарий 6. Побег из затопленного храма

```
### Сценарий: Побег из затопленного храма

Given болотный винто-ящер «Хлюп» с размахом винтов 2 м,
      максимальной скоростью 20 км/ч и энергией 40
And   ящер находится в воде

When  пилот плывёт 2 км в первую комнату
Then  энергия снижается до 38
And   ящер не в воздухе

When  пилот пытается пройти через узкий проход
Then  результат False, потому что винты шире 1.5 м
```

```
When  пилот всплывает и летит 1 км
Then  ящер в воздухе
And   энергия снижается до 36

When  пилот преодолевает стену в полёте
Then  результат True
And   энергия снижается до 26

When  пилот активирует режим глубокого погружения
Then  deep_dive становится True
And   энергия снижается до 11

When  пилот ныряет на 5 км в затопленную комнату
Then  энергия снижается до 6
And   ящер не в воздухе

When  пилот пытается проплыть 3 км обратно
Then  поднимается CrocodileNoStrength, потому что в затопленной комнате
      минимальный запас — 5

When  пилот аварийно всплывает на 1 км
Then  ящер в воздухе
And   энергия снижается до 4

Then  итоговое состояние:
      position_km == 9
      energy == 4
      is_flying == True
      deep_dive == True
```

Инструкция: пример «Пилота» на python. По факту скрывает более низкоуровневую реализацию, делегируя вызовы, но с некоторыми известными параметрами...

```python
class Pilot:
    """Эмулятор пилота. Управляет крокодилом на доменном языке."""

    def __init__(self, croc):
        self._croc = croc

    def взлететь(self, distance_km=10):
        return self._croc.move(distance_km, "air")

    def лететь(self, distance_km):
        return self._croc.move(distance_km, "air")

    def нырнуть(self, distance_km=1):
        return self._croc.move(distance_km, "water")
```

```
...
```

Инструкция: кроме «Пилота» могут быть и другие объекты, которые подчеркнут возможности средства передвижения: «Шторм», «Ущелье», «Туман» и т. д. Которые могут менять среду и, соответственно, проверять корректность передвижения…

4.3 Test Case Specification

Инструкция: таблица, в которой автор проектирует покрытие до написания тестов. Каждый TC имеет уникальный ID, тип, источник в SPEC, вход и ожидание. TCS заполняется до заморозки ТЗ, проверяется ревьюерами и служит основой для кода тестов. Каждый тест-метод содержит ID своего TC в docstring.

TEST_CASES.md содержит таблицу TCS

5. Граничные случаи

Инструкция: 2–3 случая. Они должны быть проверены тестами.

- `move(0, "air")` → `ValueError`.
- `move(1, "ground")` → `RuntimeError`.
- `Refuel(1000)` при `energy = 50` → `energy == 100`, не больше.
- `move(50, "air")` при `energy = 100` → `energy == 0`, `position_km == 50`.

6. Исключения

Инструкция: сводная таблица. Разработчик и тестировщик сверяются с ней. Нужно составить собственную иерархию исключений

Исключение	Когда
`ValueError`	Пустое имя, неположительные габариты, отрицательная дистанция, неизвестный terrain/obstacle, отрицательная заправка
~~`RuntimeError`~~	~~Недостаточно энергии, попытка движения по земле~~

Инструкция: если ошибка связана с параметрами — стандартное исключение. Если с состоянием транспорта или правилами домена — доменное. Например,

Ситуация	Тип исключения	Пример

Передан неверный тип	**TypeError**	`move("abc", "air")`
Передано недопустимое значение	**ValueError**	`move(-5, "air")`
Домен: нет сил	**CrocodileNoStrength**	`move(100, "air")` при energy=5
Домен: не умеет ходить	**CrocodileCannotWalk**	`move(1, "ground")`
Домен: слишком широкий	**CrocodileTooWide**	`can_overcome("narrow_passage")`

6. Исключения и иерархия

6.1. Иерархия

```
CrazyVehicleError
 └─── CrocodileError
    ├─── CrocodileCannotWalk
    ├─── CrocodileNoStrength
    ├─── CrocodileNoWings
    └─── CrocodileTooWide
```

6.2. Таблица

Ситуация	Исключение	Сообщение
Пустое имя	ValueError	"name must not be empty"
Отрицательная масса	ValueError	"mass_kg must be positive"
Движение по земле	CrocodileCannotWalk	"crocodile cannot walk on ground"
Нехватка энергии	CrocodileNoStrength	"not enough energy: required X, have Y"
Крылья повреждены	CrocodileNoWings	"wings are damaged"
Проход слишком узкий	CrocodileTooWide	"passage too narrow for wing span"

7. Что вне рамок

Инструкция: явно перечислить, что НЕ входит в задачу. Это защищает и автора, и разработчика.

- Транспорт не размножается.
- Транспорт не взаимодействует с другими транспортами.
- Нет графики и визуализации.
- Нет сохранения состояния между запусками.
- Нет обработки внешних файлов и API.

8. Трассируемость «требование → тест»

Инструкция: таблица обязательна. Показывает, что каждое требование покрыто тестом.

Требование	Раздел ТЗ	Тест
Конструктор валидирует имя	3.2	`test_init_empty_name_raises`
Конструктор валидирует массу	3.2	`test_init_negative_mass_raises`
Полёт расходует энергию	3.4.1	`test_move_air_consumes_energy`
Вода снижает скорость	3.4.1	`test_move_water_speed_half`
Земля запрещена	3.4.1	`test_move_ground_raises`
Стена преодолевается в полёте	3.4.2	`test_overcome_wall_flying`
Заправка не превышает 100	3.4.4	`test_refuel_caps_at_100`
Статус содержит все ключи	3.4.5	`test_status_keys`

Инструкция: каждая строка разделов 3–6 должна иметь хотя бы один тест. Если строк больше, чем тестов, — ТЗ не готово.

9. Открытые вопросы

Инструкция: всё, что автор сам не до конца решил. Это честно и полезно: ревьюеры и разработчик увидят, где могут быть дыры.

- Что делать, если `move` вызывается дважды подряд?
- Должен ли транспорт автоматически приземляться?
- Считается ли вода отдельным режимом или подрежимом полёта?